



Documentation des installations et code

Projet détection du cube, détermination de
couleurs et position et orientation avec
Raspberri Pi et OpenCV

Rédigé par : Abdallah Hojaij

Master EEA parcours SME

Année Universitaire 2025-2026

Sommaire

1	Installation et configuration	2
1.1	Mise en service du Raspberry Pi	2
1.2	Test de la caméra Module 3	2
1.3	Installation d'OpenCV, Picamera2 et NumPy	2
1.4	Vérification d'OpenCV	2
1.5	Vérification de Picamera2	2
1.6	Vérification de NumPy	2
2	Accès distant avec SSH	2
3	Accès graphique avec VNC	3
4	Documentation du code	3
4.1	Acquisition de l'image	3
5	Pourquoi utiliser HSV ?	3
6	Plages HSV utilisées	4
6.1	Création d'un masque	4
6.2	Cas particulier du rouge	5
7	Recherche des contours	5
7.1	Elimination du bruit	5
7.2	Rectangle orienté	5
7.3	Calcul des coins	6
7.4	Affichage du rectangle et du centre	6
8	Affichage des résultats	6
9	Figure du résultat	6
10	Algorithme	7
A	Code Python complet	9
B	Ressources documentaires	11

1 Installation et configuration

1.1 Mise en service du Raspberry Pi

Le Raspberry Pi disposait déjà de Raspberry Pi OS installé sur la carte microSD, la première étape a donc consisté à démarrer le système, créer le compte utilisateur et vérifier la connexion au réseau.

1.2 Test de la caméra Module 3

le test a été effectué avec la commande :

```
libcamera-hello -t 5000
```

-t 5000 demande un aperçu de la camera pendant cinq secondes

1.3 Installation d'OpenCV, Picamera2 et NumPy

Les bibliothèques sont installées depuis les dépôts de Raspberry Pi OS :

```
sudo apt update  
sudo apt install -y python3-opencv python3-picamera2 python3-numpy
```

La première commande actualise la liste des paquets disponibles

1.4 Vérification d'OpenCV

```
python3 -c "import cv2; print(cv2.__version__)"
```

le terminal affiche la version d'OpenCV

1.5 Vérification de Picamera2

```
python3 -c "from picamera2 import Picamera2; print('Picamera2 OK')"
```

Résultat obtenu :

```
Picamera2 OK
```

1.6 Vérification de NumPy

```
python3 -c "import numpy as np; print(np.__version__)"
```

2 Accès distant avec SSH

Le Raspberry Pi et le PC doivent être connectés au même réseau.

La connexion SSH est réalisée depuis le PC avec :

```
ssh abdallah@172.20.10.3
```

SSH permet de contrôler le Raspberry Pi depuis un terminal, mais n'affiche pas son bureau graphique, pour cela j'ai utilisé le vnc

3 Accès graphique avec VNC

VNC a permis d'utiliser le PC comme écran du Raspberry Pi
L'activation est réalisée avec :

```
sudo raspi-config
```

Puis :

Interface Options → VNC → Enable

Le serveur VNC peut être vérifié avec :

```
sudo ss -tlnp | grep 5900
```

Le port 5900 est le port habituellement utilisé par VNC.
Sur le PC, TigerVNC Viewer a été installé avec :

```
sudo apt install -y tigervnc-viewer
```

La connexion est ensuite lancée avec :

```
vncviewer 172.20.10.3 :5900
```

Cette méthode m'a permis d'afficher les fenêtres de la camera directement sur l'ordinateur sans connecter un écran au Raspberry Pi, ce qui m'a facilité le travail.

4 Documentation du code

Dans ce code, j'ai réalisé la detection des cubes, la détermination de leurs couleurs, coordonnées (x,y) et leurs orientation, tous ces informations sont affichées sur l'image aperçu par la camera.

4.1 Acquisition de l'image

L'image est capturée avec :

```
1 frame = camera.capture_array()
```

La variable frame contient l'image complète sous forme de tableau NumPy

5 Pourquoi utiliser HSV ?

L'image RGB représente chaque pixel par trois composantes :

(R, G, B)

Mais, ces composantes dépendent fortement de la luminosité
L'espace HSV sépare :

- **H** : la teinte(couleur)
- **S** : la saturation de la couleur
- **V** : la luminosité

Dans OpenCV :

$$0 \leq H \leq 179$$

$$0 \leq S \leq 255$$

$$0 \leq V \leq 255$$

La conversion utilisée dans le programme est :

```
1 image_hsv = cv2.cvtColor(belt, cv2.COLOR_BGR2HSV)
```

Par exemple, le centre du cube rouge a donné une teinte proche de :

$$H \approx 177$$

Cette valeur correspond bien à la seconde plage rouge dans OpenCV

6 Plages HSV utilisées

TABLE 1 – Plages HSV utilisées pour la segmentation

Couleur	Plage H	Plage S	Plage V
Rouge 1	0–10	80–255	70–255
Rouge 2	170–179	80–255	70–255
Vert	35–85	80–255	80–255
Bleu	105–130	100–255	70–255
Jaune	20–35	100–255	100–255

Les valeurs minimales de saturation permettent d'exclure les zones grises ou peu colorées. Les valeurs minimales de luminosité permettent d'exclure une partie des zones sombres.

Ces seuils ont été réglés expérimentalement et peuvent être adaptés en fonction de l'éclairage de la salle à l'université.

6.1 Création d'un masque

La fonction suivante crée une image binaire :

```
1 mask_vert = cv2.inRange(image_hsv, lower_vert, upper_vert)
```

Pour chaque pixel :

- si sa valeur HSV se trouve dans la plage, le pixel devient blanc
- sinon, il devient noir

6.2 Cas particulier du rouge

Le rouge se situe au début et à la fin de l'échelle des teintes d'OpenCV, deux masques sont donc nécessaires :

```
1 mask1 = cv2.inRange(image_hsv, lower_rouge1, upper_rouge1)
2
3 mask2 = cv2.inRange(image_hsv, lower_rouge2, upper_rouge2)
```

Les deux masques sont ensuite fusionnés :

```
1 mask_rouge = mask1 + mask2
```

7 Recherche des contours

Les contours sont recherchés sur chaque masque :

```
1 contours_couleur, _ = cv2.findContours(masque,
2 cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

`cv2.RETR_EXTERNAL` conserve les contours extérieurs.

`cv2.CHAIN_APPROX_SIMPLE` simplifie les contours en conservant principalement les points nécessaires à leur description.

7.1 Elimination du bruit

Chaque contour est analysé :

```
1 area = cv2.contourArea(cnt)
2
3 if area > 500:
```

Les contours dont l'aire est inférieure à 500 pixels carrés sont considérés comme du bruit et ne sont pas affichés

7.2 Rectangle orienté

La fonction :

```
1 rect = cv2.minAreaRect(cnt)
```

calcule le plus petit rectangle orienté contenant le contour.

La variable `rect` contient :

$$\text{rect} = ((c_x, c_y), (w, h), \theta)$$

avec :

- c_x : abscisse du centre
- c_y : ordonné du centre
- w : largeur du rectangle
- h : hauteur du rectangle
- θ : angle retourné par OpenCV

Le centre et l'angle sont récupérés avec :

```
1 (cx, cy) = rect[0]
2 angle = rect[2]
```

L'angle se trouvait principalement entre 0 et 90 degrés, car le carré est symétrique tous les 90 degrés.

7.3 Calcul des coins

```
1 box = cv2.boxPoints(rect)
2 box = box.astype(int)
```

`boxPoints()` retourne les coordonnées des quatre coins du rectangle orienté

Les coordonnées sont initialement décimales, elles sont converties en entiers avec `astype(int)` pour pouvoir les dessiner sur l'image.

7.4 Affichage du rectangle et du centre

Le rectangle est dessiné avec :

```
1 cv2.drawContours(belt, [box], 0, couleur_dessin, 2)
```

Le centre (cercle blanc) est dessiné par :

```
1 cv2.circle(belt, (int(cx), int(cy)), 4, (255, 255, 255), -1)
```

Le rayon du cercle est de quatre pixels

La valeur -1 indique que le cercle doit être rempli.

8 Affichage des résultats

Le texte est construit avec :

```
1 text = (f"{nom_couleur} " f"X:{int(cx)} " f"Y:{int(cy)} "
2         f"Angle:{angle:.1f}")
```

Le texte est écrit sur l'image avec :

```
1 cv2.putText(belt, text, (int(cx) - 60, int(cy) - 20),
2             cv2.FONT_HERSHEY_SIMPLEX, 0.45, couleur_dessin, 2)
```

9 Figure du résultat



10 Algorithme

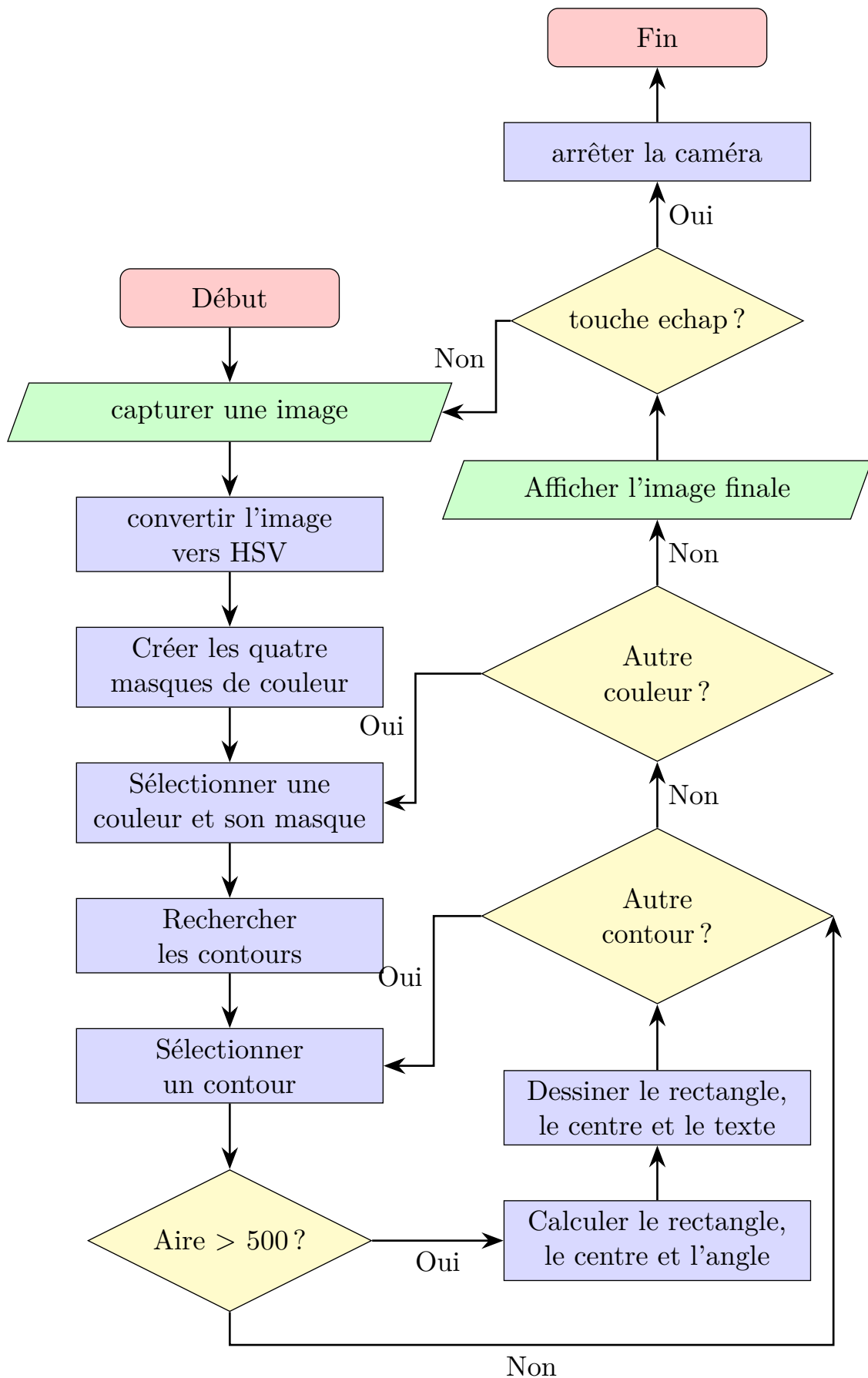


FIGURE 1 – Algorithme

A Code Python complet

```

1 from picamera2 import Picamera2
2 import numpy as np
3 import cv2
4
5 camera = Picamera2()
6
7 config = camera.create_preview_configuration(main={"size": (640, 480)
8         , "format": "RGB888"})
9
10 camera.configure(config)
11 camera.start()
12
13 while True:
14     frame = camera.capture_array()
15     belt = frame[22:431, 231:516]
16
17     #couleur
18     image_hsv = cv2.cvtColor(belt, cv2.COLOR_BGR2HSV)
19
20     # cv2.imshow("hsv", image_hsv)
21
22     lower_rouge1 = np.array([0, 80, 70])
23     upper_rouge1 = np.array([10, 255, 255])
24
25     lower_rouge2 = np.array([170, 80, 70])
26     upper_rouge2 = np.array([179, 255, 255])
27
28
29
30 # Vert
31     lower_vert = np.array([35,80,80])
32     upper_vert = np.array([85,255,255])
33
34 # Bleu
35     lower_bleu = np.array([105, 100, 70])
36     upper_bleu = np.array([130, 255, 255])
37
38 # Jaune
39     lower_jaune = np.array([20,100,100])
40     upper_jaune = np.array([35,255,255])
41
42
43
44     mask1 = cv2.inRange(image_hsv, lower_rouge1, upper_rouge1)
45     mask2 = cv2.inRange(image_hsv, lower_rouge2, upper_rouge2)
46
47     mask_rouge = mask1 + mask2
48
49     mask_vert = cv2.inRange(image_hsv, lower_vert, upper_vert)

```

```

50 mask_bleu = cv2.inRange(image_hsv, lower_bleu, upper_bleu)
51 mask_jaune = cv2.inRange(image_hsv, lower_jaune, upper_jaune)
52
53 couleurs = [("Rouge", mask_rouge, (0, 0, 255)),
54             ("Vert", mask_vert, (0, 255, 0)),
55             ("Jaune", mask_jaune, (0, 255, 255)),
56             ("Bleu", mask_bleu, (255, 0, 0))]
57
58 for nom_couleur, masque, couleur_dessin in couleurs:
59
60     contours_couleur, _ = cv2.findContours(
61         masque,
62         cv2.RETR_EXTERNAL,
63         cv2.CHAIN_APPROX_SIMPLE
64     )
65
66     for cnt in contours_couleur:
67
68         area = cv2.contourArea(cnt)
69
70         if area > 500:
71
72             rect = cv2.minAreaRect(cnt)
73
74             (cx, cy) = rect[0]
75             angle = rect[2]
76
77             box = cv2.boxPoints(rect)
78             box = box.astype(int)
79
80             cv2.drawContours(belt, [box], 0, couleur_dessin, 2)
81
82             cv2.circle(belt, (int(cx), int(cy)), 4, (255, 255, 255), -1)
83
84             text = (f"{nom_couleur} " f"X:{int(cx)} " f"Y:{int(cy)} "
85                    f"Angle:{angle:.1f}")
86
87             cv2.putText(belt, text, (int(cx) - 60, int(cy) - 20), cv2
88                 .FONT_HERSHEY_SIMPLEX, 0.45,
89                 couleur_dessin, 2)
90
91             cv2.imshow("Detection des cubes", belt)
92
93             if cv2.waitKey(1) == 27:
94                 break
95
96 cv2.destroyAllWindows()
97 camera.stop()

```

Listing 1 – Programme réalisé

B Ressources documentaires

Les principales ressources utilisées pour la préparation et la vérification de l'environnement sont :

- Documentation officielle de Raspberry Pi sur le logiciel caméra : https://www.raspberrypi.com/documentation/computers/camera_software.html
- Documentation officielle de la Camera Module : <https://www.raspberrypi.com/documentation/accessories/camera.html>
- Dépôt officiel de Picamera2 : <https://github.com/raspberrypi/picamera2>
- Documentation officielle sur l'accès distant : <https://www.raspberrypi.com/documentation/remote-access/>
- Documentation d'OpenCV : <https://docs.opencv.org/>